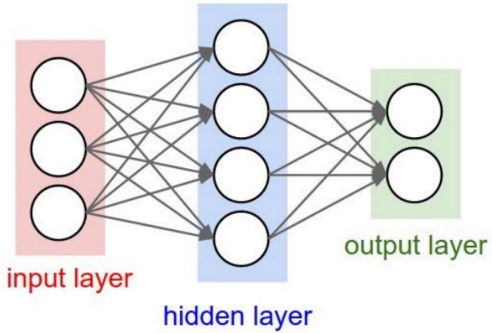
Atomic Units of Language

CSCI 601-471/671 (NLP: Self-Supervised Models)

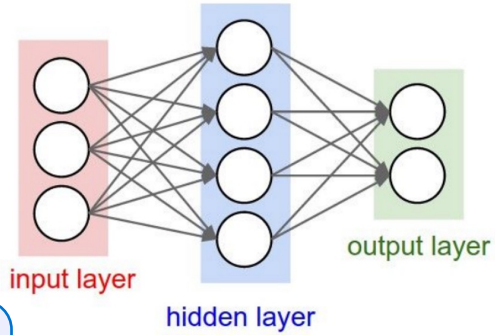
https://self-supervised.cs.jhu.edu/fa2026/

Recap: Neural Nets

- A powerful function-approximation tool.
- Can be trained efficiently via Backpropagation.
- Out focus here: how to use NNs for language modeling.



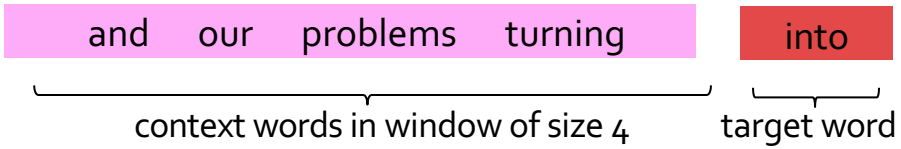
Big Picture: Language Modeling + NNs



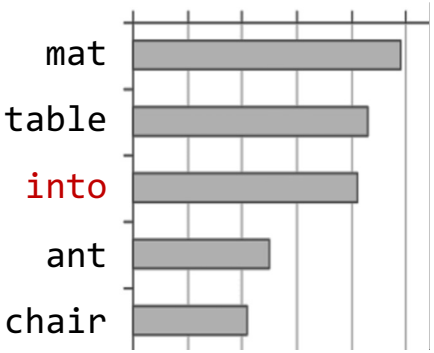
How do MLPs are for Language modeling?

Remember NNs expect numbers. How do you feed a string to neural net?

$$f(x, \Theta)$$



Probs over vocabulary



Now: Atomic Units of Language

Chapter goal: Get more familiar with different tokenization schemes and their trade-offs.

Atomic Units of Language

The cat sat on the mat.

The cat sat on the mat.

words split based on white space?

BOS, The, cat, sat, on, the, mat, ., EOS

characters?

BOS, T, h, e, SPACE, c, a, t, SPACE, s, …

bytes??!

01100001011100000111000001101100011001010110000111100000111000001101100011001010111000010111000 …

The cat sat on the mat.

words split based on white space?

BOS

chara

BOS

Which one should we use as the atomic building blocks for modeling language? 🤔

bytes??!

```
01100001011100000111000001101100011001010110000 1
11100000111000001101100011001010110000 10111000 …
```

Cost of Using Word Units

- What happens when we encounter a word at test time that we've never seen in our training data?
  - *Loquacious:* Tending to talk a great deal; talkative.
  - *Omnishambles:* A situation that has been mismanaged, due to blunders and miscalculations.
  - *COVID-19:* was unseen until 2020!
  - Acknowleadgement: incorrect spelling of "Acknowledgement"
- What about relevant words?: "dog" vs "dogs"; "run" vs "running"
- We would need a very large vocabulary to capture common words in a language.
  - Very large vocabulary size makes training difficult
- What happens with words that we haven't seen before?
  - With word level tokenization, we have no way of understanding an unseen word!
  - Also, not all languages have spaces between words like English!

Cost of Using Character Units

- What if we use characters?

- **Pro:**
  - (1) small vocabulary, just the number of unique characters in the training data.
  - (2) fewer out-of-vocabulary tokens

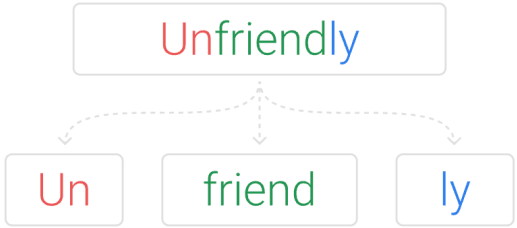
- **Cost:** much longer input sequences
  - As we discussed, modeling long-range dependences is very challenging.
  - Representing long sequences is computationally costly.

a	→	1
b	→	2
c	→	3
d	→	4
e	→	5
f	→	6
g	→	7
…	→	…
1	→	27
2	→	28
3	→	29
…	→	…
!	→	37
…		…
à	→	256

the	→	1
of	→	2
and	→	3
to	→	4
in	→	5
was	→	6
the	→	7
is	→	8
for	→	9
as	→	10
on	→	11
with	→	12
that	→	13
…		…
malapropism	→	170,000

Subword Tokenization: A Middle Ground

- Breaks words into smaller units that are indicative of their morphological construction.
  - Developed for machine translation (Sennrich et al. 2016)



- Subword tokenization is the best of both worlds
  - Common words are preserved in the vocabulary
  - Less common words are broken down into sub-words
  - This handles the problem of unseen words and large vocabulary size

- Dominantly used in modern language models (BERT, T5, GPT, …)
- Relies on a simple algorithm called Byte Pair Encoding (Gage, 1994)

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016]

[A new algorithm for data compression, Gage 1994]

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
sequence = "Using a Transformer network is simple"
print(tokenizer.tokenize(sequence))

['Using', 'a', ‘Transform', '##er', 'network', 'is', 'simple’]
```

```
print(tokenizer.convert_tokens_to_ids(tokens))

[7993, 170, 13809, 23763, 2443, 1110, 3014]
```

```
tokenizer = AutoTokenizer.from_pretrained("albert-base-v1")
sequence = "Using a Transformer network is simple"
print(tokenizer.tokenize(sequence))

[‘_using', ‘_a', ‘_transform', 'er', ‘_network', ‘_is', ‘_simple’]
```

GPT4’s Tokenizer

OpenAI's large language models (sometimes referred to as GPT's) process text using tokens, which are common sequences of characters found in a set of text. The models learn to understand the statistical relationships between these tokens, and excel at producing the next token in a sequence of tokens.

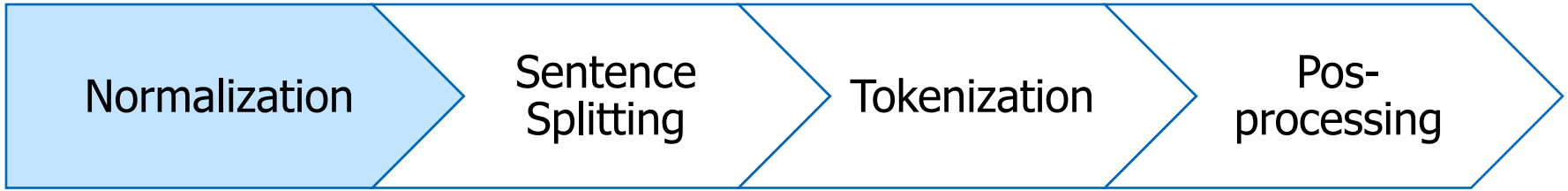
You can use the tool below to understand how a piece of text might be tokenized by a language model, and the total count of tokens in that piece of text.

It's important to note that the exact tokenization process varies between models. Newer models like GPT-3.5 and GPT-4 use a different tokenizer than our legacy GPT-3 and Codex models, and will produce different tokens for the same input text.

Here is a math problem: 234566+64432 / (33345) * 0.1234

https://platform.openai.com/tokenizer

The Tokenization Pipeline



- Converts text into a standard format to reduce variability.
  - Strip extra white spaces between words and sentences
  - Removing punctuations (“Hello, world!” → “Hello world”)
  - Unicode normalization (more on this in a sec),
  - …

Unicode normalization

- There are many characters that are different in Unicode but look very similar:

Roman "A"	A	U+0041
Fullwidth "A"	A	U+FF21

- There are various Normalization Forms to collapse visually/semantically-duplicate encodings (fullwidth vs ASCII, ligatures, old typographic variants) into one token.

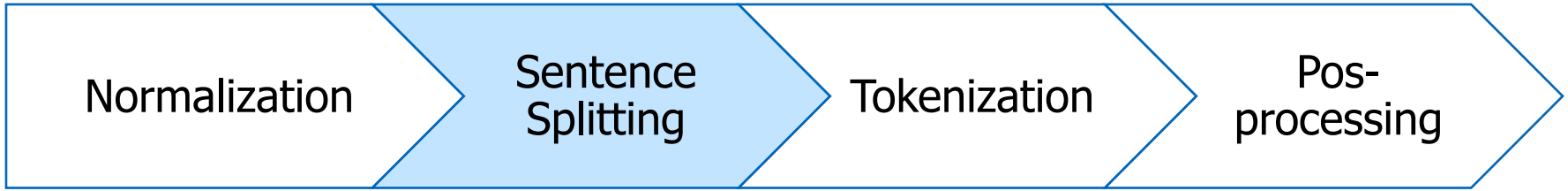
Positional variant forms	ﻉ	→	ع
	ﻊ	→	ع
	ﻋ	→	ع
	ﻌ	→	ع

Circled variants	①	→	1
Width variants	ｶ	→	力
Fractions	¼	→	1/4
Other	dž	→	dž

- NFKC normalization is a double-edged sword:
  - Benefit: shrinks vocabs size (and more compact model)
  - Cost: it's lossy and not reversible.

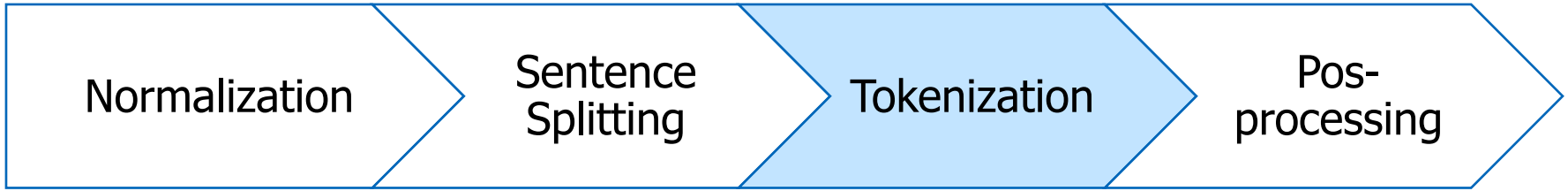
See UAX #15 for more examples and details: https://www.unicode.org/reports/tr15/

The Tokenization Pipeline



- Divides text into individual sentences.
- Uses punctuation marks (., !, ?) and language-specific rules to identify boundaries.

The Tokenization Pipeline



- Splits sentences into words or subwords.
  - Hello world → [Hello, world]
  - Map words to subwords (will discuss this in a second)
    - Trie (prefix tree) are used to store the lexicon.
    - At inference-time (tokenization) we perform fast prefix matching via Trie.

Byte-pair Encoding (BPE) – An Example

- An algorithm for forming subword tokens based on a collection of raw text.

```
and there are no refueling stations anywhere
one of the city's more unprincipled real state agents
```

Note that to do this tokenization, I need to learn it by seeing lots of text. (similar to model training)

BPE-based tokenization

```
[and, there, are, no, re, ##fueling, stations, anywhere,
one, of, the, city, 's, more, un, ##princi, ##pled, real,
state, agents]
```

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

Byte-pair Encoding (BPE) Training

Overview:

- We are given a **large** text corpus of text.
- Start by character-based tokenization.
- Repeatedly merge the most frequent adjacent tokens

- Doing 30k merges => vocabulary of around 30k subwords. Includes many whole words.

Algorithm 1 Byte-pair encoding (Sennrich et al., 2016; Gage, 1994)

```
Input: set of strings D, target vocab size k
procedure BPE(D, k)
    V ← all unique characters in D
          (about 4,000 in English Wikipedia)
    while |V| < k do                ▷ Merge tokens
        t_L, t_R ← Most frequent bigram in D
        t_NEW ← t_L + t_R           ▷ Make new token
        V ← V + [t_NEW]
        Replace each occurrence of t_L, t_R in
            D with t_NEW
    end while
    return V
end procedure
```

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example

- Form base vocabulary of all characters that occur in the training set.
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u`
Tokenized data: `j h u j h u j h u h o p k i n s h o p h o p s h o p s`

Does not show the word separator for simplicity.

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (2)

- Count the frequency of each token pair in the data
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u`
Tokenized data: `j h u j h u j h u h o p k i n s h o p h o p s h o p s`
Token pair frequencies:

- j + h -> 3
- h + u -> 3
- h + o -> 4
- o + p -> 4
- p + k -> 1
- k + i -> 1
- ....

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (3)

- Choose the pair that occurs more, merge them and add to vocab.
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u`
Tokenized data: `j h u j h u j h u h o p k i n s h o p h o p s h o p s`
Token pair frequencies:

- j + h -> 3
- h + u -> 3
- h + o -> 4 
- o + p -> 4
- p + k -> 1
- k + i -> 1
- ....

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (4)

- Choose the pair that occurs more, merge them and add to vocab.
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho` ⬅
Tokenized data: `j h u j h u j h u h o p k i n s h o p h o p s h o p s`
Token pair frequencies:

- j + h -> 3
- h + u -> 3
- h + o -> 4 ⬅
- o + p -> 4
- p + k -> 1
- k + i -> 1
- ....

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (5)

- Retokenize the data
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho`
Tokenized data: `j h u j h u j h u ho p k i n s ho p ho p s ho p s`
Token pair frequencies:

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (6)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho`
Tokenized data: `j h u j h u j h u ho p k i n s ho p ho p s ho p s`
Token pair frequencies:

- j + h -> 3
- h + u -> 3
- ho + p -> 4
- p + k -> 1
- k + i -> 1
- i + n -> 1
- ….

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (7)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho`
Tokenized data: `j h u j h u j h u ho p k i n s ho p ho p s ho p s`
Token pair frequencies:

- j + h -> 3
- h + u -> 3
- ho + p -> 4 ⬅
- p + k -> 1
- k + i -> 1
- i + n -> 1
- ....

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (7)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho, hop` ⬅
Tokenized data: `j h u j h u j h u ho p k i n s ho p ho p s ho p s`
Token pair frequencies:

- j + h -> 3
- h + u -> 3
- ho + p -> 4 ⬅
- p + k -> 1
- k + i -> 1
- i + n -> 1
- ....

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (7)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho, hop` ⬅
Tokenized data: `j h u j h u j h u hop k i n s hop hop s hop s` ⬅
Token pair frequencies:

- j + h -> 3
- h + u -> 3
- ho + p -> 4 ⬅
- p + k -> 1
- k + i -> 1
- i + n -> 1
- ....

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating🙄) training data: "jhu jhu jhu hopkins hop hops hops"
Base vocab: h, i, j, k, n, o, p, s, u, ho, hop
Tokenized data: j h u j h u j h u hop k i n s hop hop s hop s
Token pair frequencies:

- j + h -> 3 ⬅
- h + u -> 3
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
- ....

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho, hop, jh` ⬅
Tokenized data: `j h u j h u j h u hop k i n s hop hop s hop s`
Token pair frequencies:

- j + h -> 3 ⬅
- h + u -> 3
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
- ….

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho, hop, jh` ⬅
Tokenized data: `jh u jh u jh u hop k i n s hop hop s hop s` ⬅
Token pair frequencies:

- j + h -> 3 ⬅
- h + u -> 3
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
- ….

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho, hop, jh`
Tokenized data: `jh u jh u jh u hop k i n s hop hop s hop s`
Token pair frequencies:

- j h+u -> 3 ⬅
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
- ….

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho, hop, jh, jhu` ⬅
Tokenized data: `jh u jh u jh u hop k i n s hop hop s hop s`
Token pair frequencies:

- j h+u -> 3 ⬅
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
- ....

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating🙄) training data: "`jhu jhu jhu hopkins hop hops hops`"
Base vocab: `h, i, j, k, n, o, p, s, u, ho, hop, jh, jhu` ⬅
Tokenized data: `jhu jhu jhu hop k i n s hop hop s hop s` ⬅
Token pair frequencies:

- j h+u -> 3 ⬅
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
- ....

[Improving Neural Machine Translation Models with Monolingual Data, Sennrich et al. 2016] [A new algorithm for data compression, Gage 1994]

Limitations of BPE

- Loss of whole word semantics
  - E.g., “Understand” -> [“Under”, “stand”] -Doesn’t mean “stand beneath”!
- Language Dependency: Even though subwords helps in multiple-languages it may favor the structure of one language vs the other
  - Hard to apply to languages with non-concatenative (e.g., Arabic) morphology

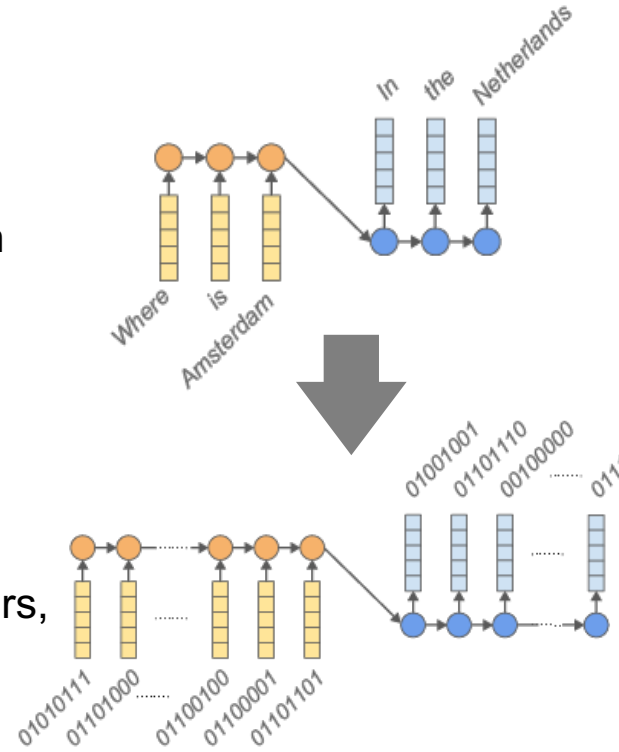
كتب	k-t-b	“write” (root form)
كَتَبَ	**k**a**t**a**b**a	“he wrote”
كَتَّبَ	**k**a**tt**a**b**a	“he made (someone) write”
اِكْتَتَبَ	i**k**ta**t**a**b**a	“he signed up”

Table 1: Non-concatenative morphology in Arabic.[4] The root contains only consonants; when conjugating, vowels, and sometimes consonants, are interleaved with the root. The root is not separable from its inflection via any contiguous split.

Clark et al., 2021, “CANINE”

Alternatives: Byte-level BPE Encoding

- Text gets encoded as UTF-8 bytes, so the base vocabulary is the 2^8 = 256 possible byte values.
  - E.g., `H -> 01010111`
  - Some works have used raw bytes as-is (e.g., ByT5) which leads to longer inputs.
- We train BPE on sequences of these bytes (i.e., merging frequently co-occurring byte pairs into new tokens).
- The final vocab (e.g., 50k tokens for GPT-2) is a mix of short byte seqs (for rare/unseen sequences) and longer byte-sequences (which end up corresponding to common characters, subwords, or whole words).
- **Pros:** (1) no out-of-vocabulary problem ever; (2) no need for a huge base vocabulary.



ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models, Xue at al. 2021, and several others]

Alternatives: WordPieces

- Merge by likelihood as measured by language model, not by frequency.
  - While voc size < target:
    1. Build a language model over your corpus
    2. Merge tokens with the highest score:

$$\text{Score} = (\text{freq\_of\_pair}) / (\text{freq\_of\_first\_element} \times \text{freq\_of\_second\_element})$$

  - **Rationale** — The algorithm prioritizes the merging of pairs where the individual parts are less frequent.
    - For example, ("un", "##able") may not merge despite its high frequency, as both parts appear frequently in corpus.
    - Conversely, ("hu", "##gging") may merge faster if “hugging” is common, as its parts are individually less frequent.

[Shuster & Nakajima 2012; Wu et al. 2016]

Alternatives: Unigram LM

- Unigram works in the other direction: it starts from a big vocabulary and removes tokens from it until it reaches the desired vocabulary size.
  - BPE/WordPieces are ‘bottom-up’ (merge characters).
  - Unigram is ‘top-down’ (prune substrings)

The algorithm computes how much the overall loss would increase if the symbol was removed and eliminates the symbols that would increase it the least.

Algorithm 2 Unigram LM (Kudo, 2018)

Input: set of strings D, target vocab size k
procedure UNIGRAMLM(D, k)
  $V \leftarrow$ all substrings occurring more than
    once in D (not crossing words)
  while $|V| > k$ **do** ▷ Prune tokens
    Fit unigram LM θ to D
    for $t \in V$ **do** ▷ Estimate token ‘loss’
      $L_t \leftarrow p_\theta(D) - p_{\theta'}(D)$
      where θ' is the LM without token t
    end for
    Remove $\min(|V| - k, \lfloor \alpha |V| \rfloor)$ of the
    tokens t with highest L_t from V,
    where $\alpha \in [0, 1]$ is a hyperparameter
  end while
  Fit final unigram LM θ to D
  return V, θ
end procedure

Alternatives: Unigram LM

Original:	furiously	**Original:**	tricycles	**Original:**	nanotechnology
BPE:	_fur iously	**BPE:**	_t ric y cles	**BPE:**	_n an ote chn ology
Uni. LM:	_fur ious ly	**Uni. LM:**	_tri cycle s	**Uni. LM:**	_nano technology

Original:	Completely preposterous suggestions
BPE:	_Comple t ely _prep ost erous _suggest ions
Unigram LM:	_Complete ly _pre post er ous _suggestion s

Original:	corrupted	**Original:**	1848 and 1852,
BPE:	_cor rupted	**BPE:**	_184 8 _and _185 2,
Unigram LM:	_corrupt ed	**Unigram LM:**	_1848 _and _1852 ,

Original	磁性は様々に分類がなされている。						
BPE	磁	性は	様々	に分類		がなされている	。
Unigram LM	磁	性	は	様々	に	分類	がなされている	。
Gloss	magnetism	(top.)	various ways	in	classification	is done	.
Translation	Magnetism is classified in various ways.						

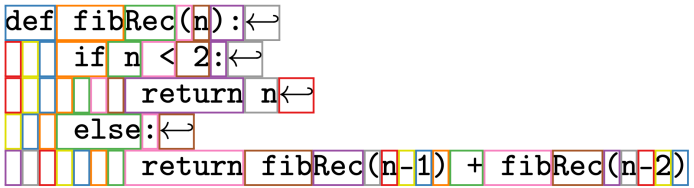
- Some (Bostrom and Durrett 2020) have argued that BPE produces less semantic tokens.
- But BPE based LMs do work fine – the transformer on top can do quite a bit.

[Slide credit: Tatsu Hashimoto]

Dealing with white spaces

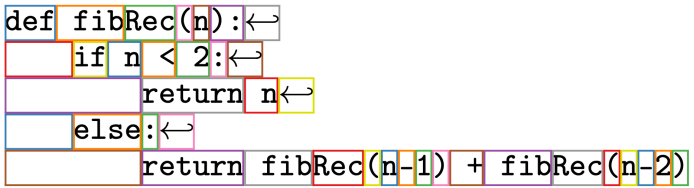
- **Why Whitespace Isn't Just "Stripped Out"**
  - Whitespace are not stripped.
  - Goal: reversibility —tokenizers must reconstruct the original text, spacing included.
- **BPE:** Often attaches space as a *prefix* to the next word ("world" vs. " world" are **different** tokens);
  - GPT-2 maps space to a visible symbol "Ġ" for uniform byte-level processing.
- The issue w/ these is long spaces since they're never merged into longer tokens.
- GPT-4's fix (cl100k_base): introduced a new pre-tokenization regex and added dedicated multi-space tokens. This groups 4 spaces into a single token and has dedicated tokens for whitespace seqs up to 128 spaces.

GPT-2



55 tokens

GPT-NeoX-20B

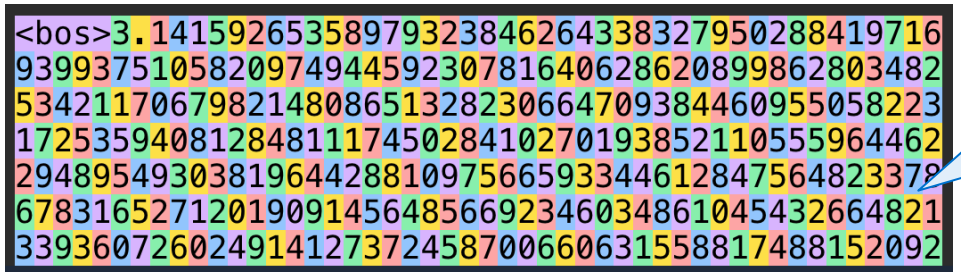


39 tokens

GPT-NeoX-20B: An Open-Source Autoregressive Language Model, 2022

Dealing with numbers

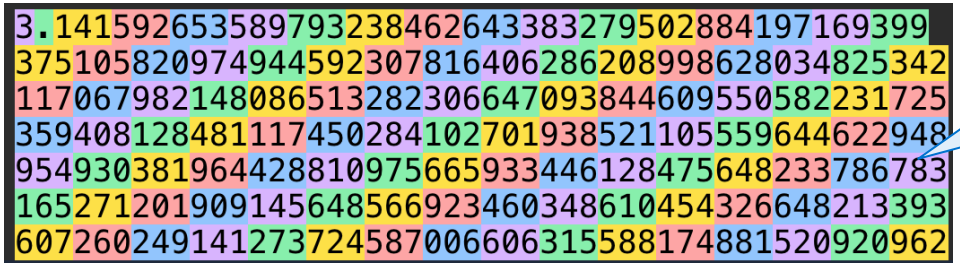
- Numbers are challenging:
  - BPE merges tokens based on frequency; digits frequency may be arbitrary.
  - Inconsistent tokenization depending on the context:
    - "1234" → 123+4, but "1235" → 12+35
    - “42” → 42 (single token; common token); “47”→ 4+7
- **Fix #1:** Force every digit (0–9) to be its own token
  - Guarantees consistency: every "7" is always the same token
  - Trade-off: longer sequences for large numbers



https://www.artfish.ai/p/how-would-you-tokenize-or-break-down

Dealing with numbers

- Numbers are challenging:
  - BPE merges tokens based on frequency; digits frequency may be arbitrary.
  - Inconsistent tokenization depending on the context:
    - "1234" → 123+4, but "1235" → 12+35
    - “42” → 42 (single token; common token); “47”→ 4+7
- **Fix #1:** Force every digit (0–9) to be its own token
  - Guarantees consistency: every "7" is always the same token
  - Trade-off: longer sequences for large numbers
- **Fix #2:** Let BPE do its merging on numbers, but cap its length (commonly 3).



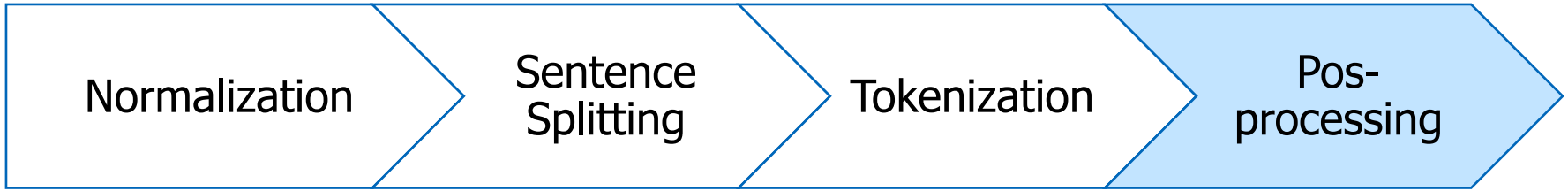
GPT-4 and **GPT-4o** tokenizers broke down numerical sequences into groups of 3.

Dealing with numbers

- Numbers are challenging:
  - BPE merges tokens based on frequency; digits frequency may be arbitrary.
  - Inconsistent tokenization depending on the context:
    - "1234" → 123+4, but "1235" → 12+35
    - “42” → 42 (single token; common token); “47”→ 4+7
- **Fix #1:** Force every digit (0–9) to be its own token
  - Guarantees consistency: every "7" is always the same token
  - Trade-off: longer sequences for large numbers
- **Fix #2:** Let BPE do its merging on numbers, but cap its length (commonly 3).
- **Fix #3:** Right-to-left chunking are shown to be more effective.
  - Chunking digits from the left doesn't align with base-10 place value
  - "1234567" → 123 | 456 | 7 (left; misaligned) vs 1 | 234 | 567

https://huggingface.co/spaces/huggingface/number-tokenization-blog

The Tokenization Pipeline



- Truncate to match the maximum length of the model
- Pad all sentences in a batch to the same length
- Add special tokens: for example:
  - <UNK> (unknown word)
  - <PAD> (padding for fixed-length sequences)
  - <BOS> (beginning of sentence)
  - <EOS> (end of sentence)
- **Finally,** converts tokens into numerical IDs for model input.
  - Uses a vocabulary (word-to-index mapping).

Special tokens

- Special tokens are **manually** inserted into the vocab by model developers.
- These are **not** meant to be used or shown by human users. They're parsed by the system that is running the LM. Hence, they use strings that're unlikely to occur naturally.
- Examples from GPT-OSS:
  - <|start|> / <|end|> : mark the **start/end of a message**
  - <|message|> : separates a message's **header (role) from its content**
  - <|channel|> : labels **which output stream** a piece of text belongs to (reasoning vs. final answer)
  - <|constrain|> : specifies the **expected format for tool-call arguments**
  - <|call|> : marks the model **invoking a tool**
  - <|return|> : marks the **final end of a response**

See OpenAI's special tokens:
https://github.com/openai/tiktoken/blob/08a5f3b2c987ada4fc5aa1f16c643c203fa8acaa/tiktoken_ext/openai_public.py#L123-L144
https://developers.openai.com/cookbook/articles/openai-harmony#special-tokens

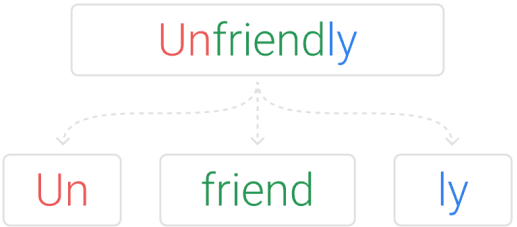
Special tokens

- Most tokenizers reserve a block of IDs up front for special tokens (e.g., Qwen reserves 208 extra empty slots).
- So new ones can be added later without resizing the whole vocabulary/embedding matrix.

Model	Release	Reserved block
Kimi K2, K2.5, K2.6	2025-26	3840
Qwen 2.5/3	2024-25	208
Gemma 3	2025	99
Llama 3	2024	256

Summary

- **Question:** what is a good atomic unit for feeding lang to LMs?
  - **Words:** too coarse
  - **Characters:** too small
  - **Subwords:** a good middle ground.



- **One nice intuition:** tokenization is compression for [almost] free.

What Tokenizers do people use?

Model	Release	Algorithm	Library
Kimi K2, K2.5, K2.6	2025-26	Byte-level BPE	tiktokens
GLM-4/GLM-4.6	2024-25	Byte-level BPE	HF tokenizers
GPT-OSS	2025	Byte-level BPE	tiktokens
Qwen 2.5/3	2024-25	Byte-level BPE	tiktokens
Gemma 3	2025	Unigram LM	SentencePieces
DeepSeek-V3	2024	Byte-level BPE	HF tokenizers

- Nearly everyone (except Google) uses BPE.
- All these tokenizers are *mostly* *invertible* (*except the ones that do aggressive normalization)

Vocab sizes

Model	Release	Vocab size
Kimi K2, K2.5, K2.6	2025-26	160k
GLM-4/GLM-4.6	2024-25	150k
GPT-OSS	2025	201k
Qwen 2.5/3	2024-25	151k
Gemma 3	2025	262k
DeepSeek-V3	2024	128k
mT5	2020	250k

Model	Release	Vocab size
Original Transformer	2017	37k
GPT1	2018	40k
GPT2/3	2019-20	50.2k
T5	2019	32k
Llama	2023	32k
GPT4	2023	100.2k

- Vocab size of English-only models (**right**) had 30-60k.
- While multi-lingual models (**left**) typically have >> 100k tokens.
- Some teams size vocabs to align with hardware/tensor-parallel efficiency.
  - 2^{15} = 32,768; 2^{16} = 65,536; 131,072 = 2^{17}; 262,144 = 2^{18};

Summary

- Everyone uses invertible subword tokenizers (BPE, Unigram) for good reason.

- What are some new ideas in tokenization?

Bonus

SuperBPE

- The standard BPE first splits the words into chunks separated by white spaces. So that a token in the vocab can *never cross the white space border*.

- What if tokens contained whitespace?
  - Multi-word expressions act as single semantic units (e.g., "by the way", “for example”)
  - Longer tokens means model can generate more efficiently.

- **Approach:**
  - **Stage 1:** learn ordinary subwords (like normal BPE)
  - **Stage 2:** learn "superwords" that span multiple words/whitespace

- **Critique:** Only works for languages for many white spaces (e.g. English). What about Chinese, Japanese and other no-white-space languages?

SuperBPE: Space Travel for Language Models, 2025

SuperBPE

Bonus

POS tag	#	Example Tokens
NN, IN	906	_case_of, _hint_of, _availability_of, _emphasis_on, _distinction_between
VB, DT	566	_reached_a, _discovered_the, _identify_the, _becomes_a, _issued_a
DT, NN	498	_this_month, _no_idea, _the_earth, _the_maximum, _this_stuff
IN, NN	406	_on_top, _by_accident, _in_effect, _for_lunch, _in_front
IN, DT	379	_on_the, _without_a, _alongside_the, _for_each
IN, DT, NN	333	_for_a_living, _by_the_way, _into_the_future, _in_the_midst
NN, IN, DT	270	_position_of_the, _component_of_the, _review_of_the, _example_of_this
IN, DT, JJ	145	_like_any_other, _with_each_other, _for_a_short, _of_the_entire
VB, IN, DT	121	_worked_as_a, _based_on_the, _combined_with_the, _turned_into_a
IN, DT, NN, IN	33	_at_the_time_of, _in_the_presence_of, _in_the_middle_of, _in_a_way_that
,, CC, PRP, VB	20	,_and_it_was, ,_but_I_think, ,_but_I_have, ,_but_I_am
IN, DT, JJ, NN	18	_in_the_long_run, _on_the_other_hand, _for_the_first_time, _in_the_same_way

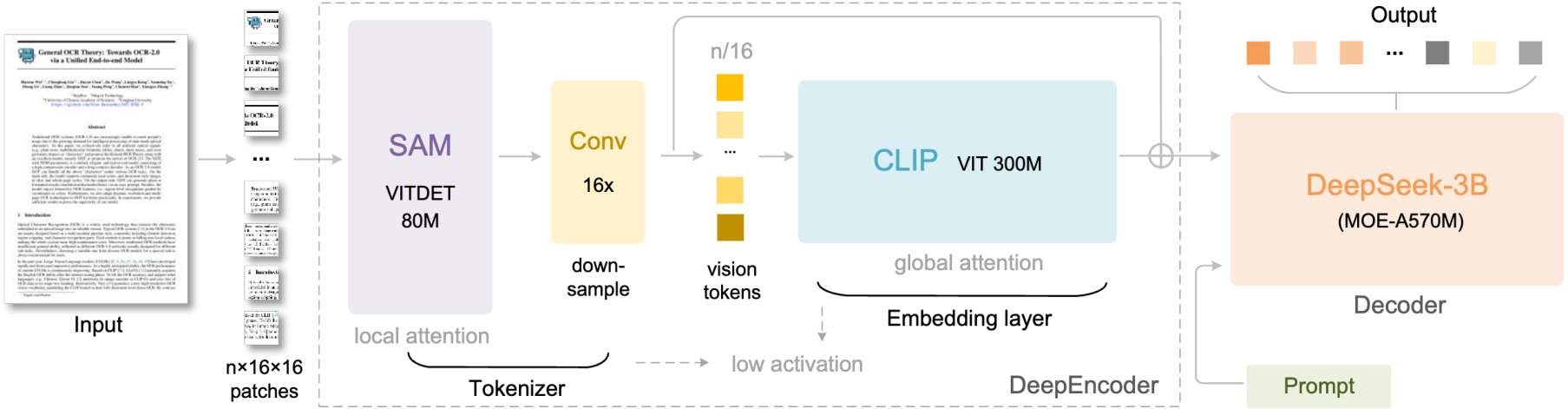
Table 3: **The most common POS tags for tokens of 2, 3, and 4 words in SuperBPE**, along with random example tokens for each tag. NN = noun, IN = preposition, VB = verb, DT = determiner, CC = conjunction, JJ = adjective, PRP = pronoun.

SuperBPE: Space Travel for Language Models, 2025

Bonus

Lang Tokenization in Visual Space

- Some prior work have tried encoding text as pixels. (Rust et al.)
- Recent works (e.g., DeepSeek-OCR) render a page as an image, and encode it into a small number of "vision tokens".
- They boast 10x compression with minimal accuracy drop.



PIXEL (Rust et al., 2022/2023); DeepSeek-OCR (Wei et al.)